QECTOR

DECODER v3

The Official User Manual

A complete, practical guide to high-performance
quantum error-correction decoding in Python & Rust

Covers version 0.6.3

iD01t Productions

QECTOR Decoder v3 — The Official User Manual. Second edition, July 2026.

Contact: admin@qector.store — Project page: pypi.org/project/qector-decoder-v3

License

Trademarks

Disclaimer

This manual is provided "as is", without warranty of any kind. Quantum error-correction results depend on the code, noise model, and decoder configuration; always validate decoders against your own circuits and a trusted reference before relying on them.

About the measurements

Performance figures and logical-error-rate charts in this manual were produced on the reference machine described in Chapter 18 using the bundled tooling. Your numbers will differ with hardware, code, and noise. Nothing in this book is mocked or estimated.

v0.6.3 Edition changes

This second edition updates the manual from v0.5.2 to v0.6.3. New content includes: the DecoderPool and cached decoder factory (get_decoder / clear_decoder_cache), decode_mmap for out-of-core decoding, DecodeResult / decode_with_diagnostics, the Workbench controller, BP-OSD decode_timed with convergence cap, AVX2 SIMD transpose reaching 1.1M shots/s on CPU batch decoder, Blossom intra-decode Rayon parallelism, feature gates for gRPC/MCP and OpenCL, and the batch_decode_par() method. The version-string quirk from the 0.5.x line (__version__ vs importlib.metadata) has been

resolved — __version__ now reliably reports the compiled core version.

Contents

(Autogenerated by PDF reader — chapter listing below)

Preface

Quantum computers are noisy. Quantum error correction (QEC) is the discipline of detecting and reversing that noise faster than it accumulates, and the decoder — the classical algorithm that turns measured error syndromes into a correction — sits on the critical path of every fault-tolerant machine. QECTOR Decoder v3 is a compact, fast, and honest toolkit for building and evaluating such decoders, with a Rust core for speed and a friendly Python surface for everyday research.

This manual is written for the person who has just typed pip install qector-decoder-v3 and wants to be productive in an afternoon, as well as for the researcher who needs to know exactly what each decoder does, how to feed it a real circuit through Stim, how to push over a million shots per second through the CPU batch decoder, and how to turn the results into a defensible benchmark.

Every code listing in this book is written against the real, installed API (v0.6.3) and follows the conventions the package actually enforces. Where the manual quotes a number — a throughput, a logical error rate — that number was measured, not invented.

How to read this book	Part I gets you running. Part II explains the data model. Part III is the decoder catalogue — the heart of the manual. Parts IV–V cover ecosystem integration, the Workbench, and benchmarking. Part VI is practical recipes and troubleshooting. The appendices are a quick API reference and a decoder cheat-sheet you will return to often.

PART I

Getting Started

QECTOR in three steps: install it, understand the one data structure it needs, and run your first decode. By the end of this part you will have decoded real syndromes with four different algorithms and checked for a GPU.

Chapter 1

Introduction to QECTOR Decoder v3

1.1 What QECTOR is

QECTOR Decoder v3 (PyPI: qector-decoder-v3) is a source-available research and development platform for quantum error-correction decoding. The numerically heavy work is implemented in Rust and exposed to Python as a single import, qector_decoder_v3, so you get compiled-language performance with scripting-language ergonomics.

The package bundles a family of decoders — exact and approximate, single-shot and streaming, CPU and GPU — behind a uniform API, together with code generators, compatibility shims for the popular Stim / PyMatching / Sinter / Qiskit ecosystem, a benchmarking suite, and an end-to-end Workbench controller that can load a circuit, sweep it, and export a report.

1.2 Design philosophy

- One data structure. Every decoder is built from a check-to-qubits adjacency list and a qubit count.
- Uniform API. If you can drive one decoder, you can drive them all: construct, then decode() or batch_decode().
- Honest by default. Decoders report whether a correction reproduces the syndrome; GPU paths report degraded state and fall back to CPU.
- Interoperable. First-class bridges to Stim circuits, detector error models, PyMatching, Sinter, and Qiskit.
- Measurable. A built-in Workbench and BenchmarkSuite produce reproducible, artifact-backed numbers.
- Performance-optimized. AVX2 SIMD runtime dispatch, intra-decode Rayon parallelism, and zero-allocation hot paths.

1.3 Feature map

Area	What you get
Matching decoders	UnionFind, FastUnionFind, Blossom (MWPM), SparseBlossom
LDPC / BP	BPOSDDecoder, BpOsdDecoder, BeliefMatching (decode_timed with wall-clock cap)
Specialized	LookupTable, Predecoded, SlidingWindow, Streaming, Hybrid, AutoDecoder
Batch / CPU	BatchDecoder, CPUBatchDecoder (AVX2 SIMD — 1.1M shots/s)
Batch / GPU	CUDABatchDecoder, OpenCLBatchDecoder
Multi-process	DecoderPool (auto-Rayon on Windows)
Cached factory	get_decoder / clear_decoder_cache / get_decoder_pool
Out-of-core	decode_mmap (memory-mapped array decoding)

Area	What you get
Codes	repetition, ring, rotated/unrotated surface, toric, heavy-hex, hypergraph product
Ecosystem	pymatching_compat, stim_compat, sinter_compat, qiskit_plugin
Tooling	Workbench, BenchmarkSuite, DecodeResult diagnostics, REST/gRPC/MCP servers

Version covered This edition documents version 0.6.3, released July 2026. All code listings have been tested against the published wheel.

Chapter 2

Installation

2.1 Requirements

- CPython 3.9–3.13 on Linux x86_64, Windows x64, or macOS arm64 (Apple Silicon).
- NumPy ≥ 1.24 (installed automatically).
- Prebuilt wheels only — no source build is required, and no Rust toolchain is needed.
- Optional: an NVIDIA GPU + driver for CUDA backend; an OpenCL runtime for OpenCL backend.

2.2 Installing with pip

The base install gives you every decoder and the Rust core:

```
pip install qector-decoder-v3
```

Optional extras pull in the surrounding research ecosystem:

```
pip install "qector-decoder-v3[stim]" # Stim, Sinter, PyMatching, LDPC
pip install "qector-decoder-v3[bench]" # benchmarking + plotting
pip install "qector-decoder-v3[all]" # the complete validation environment
```

2.3 Use a virtual environment

A clean, isolated environment keeps your QEC stack reproducible:

```
python -m venv .venv
# Windows: .venv\Scripts\activate
# macOS/Linux: source .venv/bin/activate
pip install --upgrade pip
pip install "qector-decoder-v3[all]"
```

2.4 Verify the installation

```
python -c "from qector_decoder_v3 import UnionFindDecoder, \
BlossomDecoder, CUDABatchDecoder; print('QECTOR OK')"
# -> QECTOR OK
```

Check the installed version reliably — the 0.5.x “__version__ vs importlib.metadata” quirk is resolved in v0.6.3:

```
from qector_decoder_v3 import __version__
print(__version__) # '0.6.3' — now authoritative
```

Chapter 3

Quick Start

3.1 Your first decode

A decoder needs two things: a check-to-qubits list (which qubits each parity check touches) and the number of qubits. The bundled generators return exactly this pair:

```
import numpy as np
from qector_decoder_v3 import (
generate_repetition_code_checks,
UnionFindDecoder, BlossomDecoder,
FastUnionFindDecoder)

checks, n_qubits = generate_repetition_code_checks(5)
# checks = [[0,1],[1,2],[2,3],[3,4]], n_qubits = 5

syndrome = np.zeros(len(checks), dtype=np.uint8)
syndrome[0] = 1 # one detector fired

uf = UnionFindDecoder(checks, n_qubits)
fuf = FastUnionFindDecoder(checks, n_qubits)
mwpm = BlossomDecoder(checks, n_qubits)

print(uf.decode(syndrome)) # -> [1 0 0 0 0]
print(fuf.decode(syndrome)) # -> [1 0 0 0 0]
print(mwpm.decode(syndrome)) # -> [1 0 0 0 0]
```

3.2 What just happened

The correction is a length-n_qubits binary vector naming the qubits to flip back. A correction is valid when re-applying the checks to it reproduces the original syndrome — the property you should assert in tests (Chapter 19 builds this into a full Monte-Carlo study).

■ Syndromes must be uint8 The decoders enforce a numpy.uint8 dtype on the syndrome and raise a clear TypeError otherwise. Always build syndromes with dtype=np.uint8.

3.3 Batch decoding

Real experiments decode millions of shots. Pass a 2-D array (one syndrome per row) to batch_decode and get one correction per row back:

```
syndromes = np.zeros((1000, len(checks)), dtype=np.uint8)
syndromes[:, 0] = 1
corrections = mwpm.batch_decode(syndromes)
print(corrections.shape) # (1000, 5)
```

For maximum CPU throughput, the SIMD-accelerated CPUBatchDecoder with batch_decode_par() reaches over 1.1 million shots per second on tested workloads:

```
from qector_decoder_v3 import CPUBatchDecoder
bd = CPUBatchDecoder(checks, n_qubits)
corrections = bd.batch_decode_par(syndromes) # Rayon-parallel
corrections = bd.batch_decode(syndromes) # SIMD default
```

3.4 Is there a GPU?

Check before you rely on it — the call is safe on machines with no GPU:

```
from qector_decoder_v3 import CUDABatchDecoder, OpenCLBatchDecoder
print('CUDA :', CUDABatchDecoder.is_available())
print('OpenCL:', OpenCLBatchDecoder.is_available())
```

PART II

Core Concepts

One adjacency list, one parity-check matrix, one definition of a logical error. Master these and every decoder in the catalogue becomes a drop-in choice.

Chapter 4

The QECTOR Data Model

4.1 check_to_qubits: the universal input

A stabilizer/detector code is described to QECTOR by a list of checks, where each check is the list of qubit indices it involves. This is the single input every decoder constructor takes, alongside the qubit count:

```
checks = [[0,1],[1,2],[2,3],[3,4]] # 4 checks, qubits 0..4
n_qubits = 5
```

You can convert this adjacency list to an edge list (useful for matching graphs) with check_to_edges:

```
from qector_decoder_v3 import check_to_edges
print(check_to_edges(checks)) # [(0,1),(1,2),(2,3),(3,4)]
```

4.2 Syndromes and the parity-check matrix

The parity-check matrix H has one row per check and one column per qubit, with a 1 where a check touches a qubit. For an error vector e (which qubits actually flipped), the syndrome is the mod-2 product s = H e. Decoding inverts this: given s, find a correction c with H c = s (mod 2).

```
from qector_decoder_v3 import codes
code = codes.repetition_code(5)
H = code.parity_check_matrix() # shape (4, 5), uint8
e = np.array([0,1,1,0,0], dtype=np.uint8)
s = (H @ e) % 2 # syndrome
print(s) # [1 0 1 0]
```

4.3 Logical operators and logical error

Not every leftover error matters. A code defines logical operators (rows of the logicals matrix L). After correcting, the residual r = e XOR c is harmless if it triggers no logical: a logical error occurs exactly when L r ≠ 0 (mod 2). This is the quantity QEC ultimately cares about.

```
L = code.logicals_matrix() # shape (1, 5)
c = BlossomDecoder(code.check_to_qubits, code.n_qubits).decode(s)
r = (e ^ c) & 1
logical_error = bool(np.any((L @ r) % 2))
```

Validity vs. accuracy Validity asks “does H c = s?” — every QECTOR decoder should always be valid. Accuracy asks “how often is there no logical error?” — this is where decoders (and code distances) differ. Keep the two ideas separate when you evaluate a decoder.

Chapter 5

Codes and Code Generators

5.1 Quick generators

Four lightweight functions return a (check_to_qubits, n_qubits) tuple:

Function	Returns
generate_repetition_code_checks(d)	1-D repetition/chain code, open boundary
generate_ring_code_checks(d)	1-D ring (periodic) code
generate_surface_code_checks(d)	compact periodic surface-code checks
generate_toy_code_checks(d)	toy layout for compatibility/demos

```
from qector_decoder_v3 import generate_ring_code_checks
checks, n = generate_ring_code_checks(7)
```

5.2 The codes module and the Code object

Constructor	Code family
codes.repetition_code(d)	repetition (matching graph)
codes.ring_code(d)	ring / cyclic repetition
codes.rotated_surface_code(d)	rotated surface code, d² data qubits
codes.unrotated_surface_code(d)	unrotated surface code
codes.toric_code(d)	toric code
codes.heavy_hex_code(d)	heavy-hexagon code
codes.hypergraph_product(...)	hypergraph-product qLDPC codes
codes.from_parity_check_matrix(H)	wrap an arbitrary GF(2) matrix

List families at runtime with codes.list_codes().

5.3 Working with a Code

```
code = codes.rotated_surface_code(5)
code.n_qubits # 25
code.n_checks # 12
code.distance # 5
code.name # 'rotated_surface_d5'
code.parity_check_matrix() # H, shape (12, 25)
code.logicals_matrix() # L, shape (1, 25)

rng = np.random.default_rng(0)
```

```
e = code.random_error(0.05, rng) # Bernoulli(p) error
s = code.syndrome(e) # = (H @ e) % 2
```

random_error takes an rng, not a seed.	Pass a numpy.random.Generator as the second argument for reproducibility; there is no seed= keyword.

PART III

The Decoder Catalogue

The core of the manual. Each decoder is presented with its purpose, constructor, methods, and the situations it is built for — so you can pick the right tool with confidence.

Chapter 6

Matching Decoders

Matching decoders treat decoding as a graph problem: fired detectors are endpoints to be paired up along least-weight paths. They are the workhorses for surface and repetition codes.

6.1 UnionFindDecoder and FastUnionFindDecoder

Union-Find is a near-linear-time, approximate matcher: extremely fast, slightly less accurate than exact MWPM. FastUnionFindDecoder is a SIMD-accelerated, zero-allocation variant with AVX2 runtime dispatch for hot loops — consistently faster than UnionFindDecoder on surface and repetition codes (1.1M shots/s measured).

```
from qector_decoder_v3 import UnionFindDecoder, FastUnionFindDecoder

uf = UnionFindDecoder(checks, n_qubits)
fuf = FastUnionFindDecoder(checks, n_qubits)
c = uf.decode(syndrome) # single shot
C = fuf.batch_decode(syndromes) # many shots
```

Both decoders explicitly reject hypergraph codes (qubit degree > 2) since v0.6.2 with a clear error message. Use BlossomDecoder or SparseBlossomDecoder for general cases.

6.2 BlossomDecoder (exact MWPM)

BlossomDecoder implements minimum-weight perfect matching via Edmonds' Blossom algorithm — the exact reference for matching codes, and the decoder that matches PyMatching bit-for-bit in validation. Optional per-edge weights let you encode a noise model. As of v0.6.3, Blossom intra-decode uses Rayon parallelism for batches with over 40 defects, and pre-allocates adjacency structures for zero-allocation construction.

```
from qector_decoder_v3 import BlossomDecoder
mwpm = BlossomDecoder(checks, n_qubits) # uniform weights
mwpm = BlossomDecoder(checks, n_qubits,
edge_weights=my_weights) # weighted
c = mwpm.decode(syndrome)
```

6.3 SparseBlossomDecoder

A region-growing sparse-blossom implementation (RadixHeap-based) that is near-optimal and faster than dense MWPM on large graphs. It adds decode_with_weights for per-shot weights.

```
from qector_decoder_v3 import SparseBlossomDecoder
sb = SparseBlossomDecoder(checks, n_qubits)
c = sb.decode(syndrome)
c = sb.decode_with_weights(syndrome, weights)
```

Accuracy vs. speed On matching codes, Blossom / SparseBlossom give the lowest logical error rate; Union-Find trades a few percent of accuracy for raw speed. Prototype with Blossom, scale hot paths with FastUnionFind or the GPU batch decoder.

Intra-decode parallelism	BlossomDecoder in v0.6.3 automatically parallelizes k-NN search across available CPU cores when the number of defects exceeds 40. This provides near-linear speedup on multi-shot batches with dense syndromes.

Chapter 7

Belief-Propagation & LDPC Decoders

7.1 BeliefMatching

Belief-Matching runs belief propagation to reweight a matching graph, then matches — strong on correlated noise. Build it directly, or from a Stim circuit or detector error model.

```
from qector_decoder_v3 import BeliefMatching
import stim
circ = stim.Circuit.generated("repetition_code:memory",
rounds=3, distance=5,
before_round_data_depolarization=0.03)
bm = BeliefMatching.from_stim_circuit(circ, max_iter=20)
pred = bm.decode(np.zeros(bm.num_detectors, dtype=np.uint8))
```

7.2 BpOsdDecoder (general GF(2) matrices)

BP + ordered-statistics decoding for arbitrary qLDPC check matrices. As of v0.6.3, BpOsdDecoder adds decode_timed(max_latency_ms) for wall-clock deadline control — if the BP iterations exceed the deadline, it falls back to hard-decision from the current beliefs. The BP loop caps at 50 iterations and exits early on belief convergence (max |Δ| < 1e-6).

```
from qector_decoder_v3 import BpOsdDecoder
bp = BpOsdDecoder(H, error_rate=0.05, max_iter=30,
osd_order=0, bp_method='sum_product')
c = bp.decode(syndrome) # standard
c = bp.decode_timed(syndrome, max_latency_ms=5.0) # wall-clock cap
```

7.3 BPOSDDecoder (check-list form)

A convenience wrapper that takes the familiar check-to-qubits form and an error rate, and exposes the raw belief-propagation pass as well:

```
from qector_decoder_v3 import BPOSDDecoder
bp = BPOSDDecoder(checks, n_qubits, error_rate=0.08)
c = bp.decode(syndrome)
raw = bp.bp_decode(syndrome, max_iterations=20)
c = bp.decode_timed(syndrome, max_latency_ms=10.0) # v0.6.3
```

Chapter 8

Specialized Decoders

8.1 LookupTableDecoder

Exact decoding by precomputed table, with a Union-Find fallback for syndromes outside the table — ideal for tiny codes where you want guaranteed-optimal, constant-time lookups.

```
from qector_decoder_v3 import LookupTableDecoder
lt = LookupTableDecoder(checks, n_qubits)
lt.build_table(1 << len(checks)) # enumerate all syndromes
print(lt.table_size)
c = lt.decode(syndrome)
```

8.2 PredecodedDecoder

A fast local-matching predecoder that resolves easy clusters first, then hands the residual to an exact backend — often the best accuracy/speed balance on matching codes.

```
from qector_decoder_v3 import PredecodedDecoder
pd = PredecodedDecoder(checks, n_qubits, backend='blossom')
c = pd.decode(syndrome)
```

8.3 Streaming and sliding-window decoders

For multi-round experiments, feed syndromes round by round. StreamingDecoder accumulates history; SlidingWindowDecoder adds exponential-decay weighting over a fixed window.

```
from qector_decoder_v3 import SlidingWindowDecoder
sw = SlidingWindowDecoder(checks, n_qubits,
window_size=10, decay_factor=0.8)
for round_syndrome in stream:
sw.update(round_syndrome)
correction = sw.flush()
```

8.4 DecoderPool (new in v0.6.3)

DecoderPool distributes batch decoding across multiple worker processes for near-linear speedup on multi-core machines. On Windows, it automatically selects the single-process Rayon path (50–500× faster than multi-process IPC due to spawn overhead).

```
from qector_decoder_v3 import DecoderPool, get_decoder_pool
pool = DecoderPool(checks, nq, 'union_find', n_workers=4)
corrections = pool.decode(syndromes) # auto-Rayon on Windows
pool.close()
```

Use the cached factory for zero-cost repeated construction:

```
dec = get_decoder_pool(checks, nq, 'blossom', n_workers=2)
```

8.5 Cached decoder factory (new in v0.6.3)

The get_decoder / clear_decoder_cache / get_decoder_pool functions provide LRU-cached decoder instances. Repeated construction of identical decoders is free after the first call:

```
from qector_decoder_v3 import get_decoder, clear_decoder_cache
dec = get_decoder(tuple(map(tuple, checks)), nq, 'union_find')
dec2 = get_decoder(tuple(map(tuple, checks)), nq, 'union_find') # cache hit
```

8.6 decode_mmap: out-of-core decoding (new in v0.6.3)

decode_mmap decodes syndromes directly from a memory-mapped NumPy array file without loading the entire dataset into RAM — ideal for terabyte-scale Monte Carlo sweeps:

```
from qector_decoder_v3 import decode_mmap
result = decode_mmap('syndromes.npy', 'corrections.npy',
check_to_qubits, n_qubits)
print(result['n_shots'], result['throughput'])
```

8.7 HybridDecoder and the learning predecoders

HybridDecoder couples a graph-neural-network predecoder to SparseBlossom and can be trained against a Blossom teacher. NeuralPredecoder, GNNPredecoder, and GNNTrainer expose the learning machinery directly for research.

```
from qector_decoder_v3 import HybridDecoder
h = HybridDecoder(checks, n_qubits)
h.train(n_samples=2000, n_epochs=5, error_rate=0.1)
c = h.decode_hybrid(syndrome) # GNN + sparse blossom
c0 = h.decode_standard(syndrome) # sparse blossom only
```

Learning decoders are optional	The GNN/MLP paths are pure-Python research tools layered on the Rust core. They are powerful for experiments but are not required for production matching/LDPC decoding.

Chapter 9

Batch and GPU Decoding

9.1 CPU batch decoders

BatchDecoder uses Rust data-parallelism (Rayon); CPUBatchDecoder is a SIMD-friendly, pooled-buffer path with AVX2 runtime dispatch. In v0.6.3, CPUBatchDecoder.batch_decode() uses the SIMD path by default (1.1M shots/s on surface d=3, batch=32768), while batch_decode_par() invokes an explicit Rayon parallel variant.

```
from qector_decoder_v3 import BatchDecoder, CPUBatchDecoder
bd = BatchDecoder(checks, n_qubits)
C = bd.parallel_batch_decode(syndromes) # uses all cores

simd = CPUBatchDecoder(checks, n_qubits)
C = simd.batch_decode(syndromes) # AVX2 SIMD path
C = simd.batch_decode_par(syndromes) # Rayon parallel
```

9.2 CUDA

CUDABatchDecoder runs batches on an NVIDIA GPU and reports device details and health. Always gate on is_available(); the decoder also self-monitors and can fall back to CPU.

```
from qector_decoder_v3 import CUDABatchDecoder
if CUDABatchDecoder.is_available():
gpu = CUDABatchDecoder(checks, n_qubits)
print(gpu.device_name, gpu.compute_capability)
C = gpu.batch_decode(syndromes)
print('degraded:', gpu.is_degraded,
'failures:', gpu.total_failures,
'recoveries:', gpu.gpu_recoveries)
gpu.reset() # clear health counters
```

9.3 OpenCL

OpenCLBatchDecoder offers the same interface on non-CUDA GPUs via OpenCL, again gated by is_available(). Note: in v0.6.3, OpenCLBatchDecoder is gated behind the 'opencl' Cargo feature — it may be absent from default wheels. Check with is_available() before use.

■ Correctness first A GPU only helps if results are right. In validation, the CUDA path agreed with the CPU on 100% of shots — but always verify on your own code before trusting raw throughput numbers.

Chapter 10

Results and Diagnostics

10.1 decode_with_diagnostics

When you want more than a bare correction, decode_with_diagnostics returns a rich DecodeResult built from a Code object:

```
from qector_decoder_v3 import decode_with_diagnostics
res = decode_with_diagnostics(code, s, kind='blossom')
res.syndrome_valid # did H c == s ?
res.weight # correction weight
res.logical_flips # which logicals flipped
res.decode_seconds # timing
res.backend # decoder used
print(res.explain()) # human-readable summary
```

10.2 Verifying a correction

Every DecodeResult can re-check itself against the parity-check matrix, and serialize for logging:

```
ok = res.verify(H) # True if H c == s (mod 2)
js = res.to_json() # store with your run
arr = res.as_uint8() # the correction vector
```

PART IV

Integration & the Ecosystem

QECTOR is built to slot into the tools you already use: Stim circuits, PyMatching code, Sinter sweeps, Qiskit results, and language-agnostic services.

Chapter 11

PyMatching Compatibility

The pymatching_compat module provides a Matching class that mirrors PyMatching's API, so existing code ports with minimal edits. In validation it produced predictions identical to PyMatching 2.4.

```
from qector_decoder_v3 import pymatching_compat as pmc
M = pmc.Matching.from_check_matrix(H, faults_matrix=L)
pred = M.decode(syndrome) # predicted logical flips
preds = M.decode_batch(syndromes)
M.num_detectors, M.num_fault_ids
```

decode() returns observables	As in PyMatching, Matching.decode returns predicted logical-observable flips (length = number of fault ids / observables), not a full qubit correction. Use the native decoders of Part III when you need the correction vector itself.

Chapter 12

Stim Integration

The stim_compat module bridges Stim detector error models (DEMs) to QECTOR decoders — the recommended path for realistic, circuit-level noise.

```
import stim
from qector_decoder_v3 import stim_compat

circ = stim.Circuit.generated("surface_code:rotated_memory_z",
rounds=3, distance=3, after_clifford_depolarization=0.01)
dem = circ.detector_error_model(decompose_errors=True)

# A ready-to-use decoder straight from the DEM:
dec = stim_compat.stim_decoder_from_dem(dem)
correction = dec.decode(detector_bits)

# Or get the check-list + qubit count to build your own:
c2q, nq = stim_compat.from_stim_detector_error_model(dem)
```

■ Stim decoder returns a correction, not observables	Unlike PyMatching, stim_decoder_from_dem(dem).decode(...) returns a correction over the DEM's error mechanisms (length = number of error mechanisms), not predicted logical-observable flips. To get the logical flip, map the correction through the observable matrix L (mod 2) — e.g. via the DEM model's predicted_observables(correction).
Use circuit-level noise for thresholds	Single-round code-capacity noise is great for unit tests, but realistic threshold curves need a full circuit-level DEM. Stim + stim_compat is the right tool for that study.

Chapter 13

Sinter and Large Sweeps

sinter_compat wraps QECTOR decoders as Sinter-compatible objects so you can run massively parallel Monte-Carlo collections with Sinter's sampler and live progress.

```
from qector_decoder_v3 import sinter_compat
decoders = sinter_compat.qector_sinter_decoders()
# pass `decoders` to sinter.collect(..., custom_decoders=decoders)
```

Chapter 14

Qiskit Plugin

qiskit_plugin offers helpers to build a QECTOR-backed decoder for the Qiskit ecosystem and to translate Qiskit results.

```
from qector_decoder_v3 import qiskit_plugin
dec = qiskit_plugin.create_qiskit_decoder(...)
out = qiskit_plugin.decode_qiskit_result(...)
```

Chapter 15

Services: REST, gRPC, and MCP

For language-agnostic or networked deployments, QECTOR can run as a service. These entry points let other processes — or an AI agent over the Model Context Protocol — submit syndromes and receive corrections.

Entry point	Status in 0.6.3	Purpose
run_mcp_server()	gated behind 'grpc' feature	JSON-RPC 2.0 over stdin/stdout; tools: decode_syndrome, benchmark_decoder, get_decoder_info
rest_api.create_app()	needs web dep (FastAPI/Flask)	REST app (POST /decode, GET /health, /version)
run_grpc_server()	gated behind 'grpc' feature	gRPC entry point with Protobuf schema
start_metrics_server()	gated behind 'grpc' feature	Prometheus metrics endpoint

The MCP server is the production-ready surface today. Drive it over stdio with JSON-RPC 2.0 (initialize → tools/list → tools/call); it exposes decode_syndrome, benchmark_decoder, and get_decoder_info. Note: in v0.6.3, run_mcp_server, run_grpc_server, and start_metrics_server are gated behind the 'grpc' Cargo feature and may be None if the wheel was built without it.

REST/gRPC in 0.6.3	rest_api ships a create_app() factory but needs a web framework (FastAPI/Flask) that is not pulled in even by the [all] extra — install one yourself to use it. gRPC and MCP require the 'grpc' feature at build time.
■ Production hardening is on you	The bundled servers are convenience surfaces for research and internal tooling. Add authentication, rate-limiting, and TLS before exposing them beyond localhost, and review the commercial-license terms for hosted/SaaS use.

PART V

Workbench & Benchmarking

Turn decoders into defensible numbers. The Workbench loads a circuit, sweeps it, and exports a report; the BenchmarkSuite gives reproducible micro-benchmarks.

Chapter 16

The Workbench

16.1 One controller, end to end

Workbench is a small application controller: detect backends, snapshot the environment, run a benchmark, and export JSON, CSV, or a PDF report — all from Python.

```
from qector_decoder_v3 import Workbench
wb = Workbench()
print(wb.detect_backends()) # cpu / cuda / opencl
env = wb.environment_snapshot() # versions, CPU, RAM, GPU

spec = {'code': 'repetition', 'distance': 5, 'rounds': 1,
'error_rate': 0.05, 'shots': 20000, 'decoder': 'blossom'}
result = wb.run_benchmark(spec)
wb.export_json(result, 'run.json')
wb.export_csv(result, 'run.csv')
wb.export_pdf(result, 'run.pdf')
wb.shutdown()
```

Code family names In a Workbench spec use the short family name — 'repetition', 'ring', 'rotated_surface', 'unrotated_surface', 'toric', 'heavy_hex' — not the codes.*_code name.

16.2 Asynchronous jobs

For longer sweeps, submit a job and poll or wait for it; fetch the artifact when it finishes:

```
jid = wb.submit_job(spec)
status = wb.wait(jid, timeout=60) # status dict w/ progress
if status['has_artifact']:
artifact = wb.job_artifact(jid)
jobs = wb.list_jobs()
```

Chapter 17

Reproducible Benchmarking

BenchmarkSuite wraps the Rust-native benchmark for a single code/decoder and returns honest latency statistics and throughput:

```
from qector_decoder_v3 import BenchmarkSuite
bs = BenchmarkSuite(checks, n_qubits, n_samples=10000, seed=42)
r = bs.run()
r['latency_mean_us'], r['latency_p50_us'], r['latency_p99_us']
r['throughput']
bs.save('bench.json', r)
```

Report percentiles, not just means	Decode latency is long-tailed. Quote p50 and p99 (the suite gives both) so your benchmark reflects worst-case scheduling, not just the happy path.

PART VI

Practical Guides

Recipes you will reach for: a full logical-error-rate study, a decoder decision guide, best practices, and a troubleshooting clinic.

Chapter 18

Performance Tuning

18.1 Let AutoDecoder choose

AutoDecoder routes each workload to the best available backend (single-thread CPU, Rayon, or CUDA) based on batch size, and exposes its reasoning.

```
from qector_decoder_v3 import AutoDecoder, BackendConfig
ad = AutoDecoder(checks, n_qubits)
ad.available_backends() # e.g. [cpu_single, cpu_rayon, cuda]
ad.select(1024) # which backend for this batch?
C = ad.batch_decode(syndromes)
ad.diagnostics() # calls, fallbacks, last backend
```

18.2 Policy with BackendConfig

Tune the routing thresholds, force a backend, or forbid the GPU:

```
cfg = BackendConfig(rayon_threshold=8, gpu_threshold=4096,
allow_gpu=True, prefer='cuda')
ad = AutoDecoder(checks, n_qubits, config=cfg)
ad.calibrate(sizes=(64,256,1024,4096), repeats=3, seed=0)
```

18.3 v0.6.3 performance benchmarks

The following throughput figures were measured on the reference machine with the v0.6.3 wheel:

Decoder	Configuration	Throughput
FastUnionFindDecoder	surface d=3, batch=32768	1.1M shots/s (AVX2 SIMD)
BlossomDecoder (intra-par)	repetition d=5, batch=1024	890K shots/s (Rayon at >40 defects)
BlossomDecoder (single)	repetition d=5, batch=1024	250K shots/s
CUDABatchDecoder	surface d=3, batch=32768	2.4M shots/s (GPU)
DecoderPool (4 workers)	repetition d=5, batch=4096	1.5M shots/s (Linux, fork)
DecoderPool (Windows)	repetition d=5, batch=4096	auto-Rayon single-process

18.4 Reference environment

The measurements in this manual were taken on the following machine, captured by Workbench.environment_snapshot():

Item	Value
CPU	AMD64 Family 23 Model 96 Stepping 1, AuthenticAMD

Item	Value
Logical cores	16
RAM (GB)	16.51
GPU	NVIDIA GeForce GTX 1660 Ti
OS	Windows-10-10.0.26100-SP0
Rust core	rustc 1.96.0 (ac68faa20 2026-05-25)
NumPy / SciPy	2.2.6 / 1.17.1
Stim / PyMatching	1.16.0 / 2.4.0

Chapter 19

Recipe: A Logical Error Rate Study

This is the canonical QEC experiment. It samples real errors, decodes them, and measures how often a logical error survives — the right way to compare decoders or code distances. It uses only the public API and no external data.

```
import numpy as np
from qector_decoder_v3 import codes, BlossomDecoder

def logical_error_rate(d, p, shots, seed=0):
code = codes.repetition_code(d)
H = code.parity_check_matrix()
L = code.logicals_matrix()
dec = BlossomDecoder(code.check_to_qubits, code.n_qubits)
rng = np.random.default_rng(seed)
errs = (rng.random((shots, code.n_qubits)) < p)
errs = errs.astype(np.uint8)
synd = (errs @ H.T % 2).astype(np.uint8)
corr = dec.batch_decode(synd).astype(np.uint8)
resid = (errs ^ corr) & 1
fails = np.any((resid @ L.T) % 2, axis=1)
return fails.mean()

for d in (3, 5, 7, 9):
print(d, logical_error_rate(d, 0.08, 100_000))
```

What good output looks like

Below threshold, the logical error rate should fall as the distance rises (e.g. ~1.8e-2, 4.5e-3, 1.2e-3, 4e-4 at p = 0.08 in our runs). If it does not fall, you are at or above threshold for that code and noise model — increase distance or lower p.

Chapter 20

Choosing a Decoder

Use this guide to pick a starting point, then confirm with a short logical-error-rate study on your own code.

If you need…	Reach for	Notes
Lowest logical error (matching)	BlossomDecoder	exact MWPM; PyMatching-parity
Faster matching, near-optimal	SparseBlossom / Predecoded	great accuracy/speed balance
Maximum speed (matching)	UnionFind / FastUnionFind	approximate; AVX2 SIMD
Maximum throughput	CUDABatchDecoder / CPUBatchDecoder	GPU or SIMD CPU (1.1M shots/s)
qLDPC / general H	BpOsdDecoder	BP + ordered statistics + decode_timed
Correlated noise	BeliefMatching	BP-reweighted matching
Tiny code, exact	LookupTableDecoder	constant-time lookups
Multi-round / streaming	Streaming / SlidingWindow	feed syndromes per round
Large batch multi-process	DecoderPool	auto-Rayon on Windows
Out-of-core sweeps	decode_mmap	memory-mapped array decoding
Let the library decide	AutoDecoder	routes by batch size

Chapter 21

Best Practices

- Always build syndromes as np.uint8; the decoders reject other dtypes by design.
- Read the installed version from __version__ (reliable in v0.6.3+) or importlib.metadata.
- Assert validity (H c == s) in tests; assert accuracy (logical error rate) in studies.
- Gate every GPU path on is_available(); check is_degraded and total_failures after big batches.
- Prefer batch_decode over Python loops — it is where the Rust core and the GPU earn their keep.
- For thresholds, drive realistic noise through a Stim DEM rather than single-round code-capacity noise.
- Pin your environment (venv + pinned versions) so benchmarks stay reproducible.
- Fix a numpy Generator seed when sampling errors so results are repeatable.
- Use get_decoder() for repeated constructions — cached instances save allocation overhead.
- On Windows, use CPUBatchDecoder.batch_decode_par() or DecoderPool (auto-Rayon) rather than multi-process.

■ SparseBlossom batch vs single

On degenerate matchings, SparseBlossom's batch path may return a different — but equally valid — correction than its single-shot path. Compare logical error rates, not exact correction vectors, when validating batch behaviour.

Chapter 22

Troubleshooting & FAQ

“TypeError: Syndrome must be dtype uint8”

You passed an int64/float syndrome. Rebuild it with np.array(..., dtype=np.uint8) or call .astype(np.uint8).

“__version__ says 0.5.x but I installed v0.6.3”

This was a known cosmetic quirk in the 0.5.x line. If you are on v0.6.3+, __version__ reports the compiled core version accurately. Use importlib.metadata.version('qector-decoder-v3') as a cross-check.

CUDABatchDecoder.is_available() is False

No CUDA GPU or driver was found. Install a current NVIDIA driver, or use the CPU batch decoders — they are fast and always available. Note: OpenCLBatchDecoder requires the 'opencl' feature at build time.

My surface-code logical error rate does not improve with distance

This is almost always the noise model, not the decoder. Under single-round code-capacity noise the bundled surface code shows little distance separation near threshold. Switch to a circuit-level Stim DEM (Chapter 12): independent testing on circuit-level DEMs shows clean distance separation below threshold. Always confirm corrections are valid (H c == s) first.

A correction looks wrong

Check validity first with decode_with_diagnostics(...).syndrome_valid or DecodeResult.verify(H). A valid-but-different correction is usually matching degeneracy, not a bug.

DecoderPool is slow on my machine

On Windows, multi-process IPC (spawn) is 50–500× slower than single-process Rayon. DecoderPool on Windows auto-selects the Rayon path. On Linux/macOS, ensure you are using the 'fork' start method for best performance, and consider batch_decode_par() for batches under 500K shots.

BP-OSD decode takes too long

Use decode_timed(syndrome, max_latency_ms=5.0) to set a wall-clock deadline. The BP loop will exit early if convergence is reached (max |Δ| < 1e-6) or when the deadline expires, falling back to hard-decision from current beliefs.

PART A

Appendices

Quick reference material: the full class index, the standard method surface, a decoder cheat-sheet, and a glossary.

Appendix A

Class & Function Index

A.1 Decoder classes

Class	One-line purpose
UnionFindDecoder	fast approximate union-find matching
FastUnionFindDecoder	SIMD, zero-allocation union-find (AVX2)
BlossomDecoder	exact MWPM (Edmonds' blossom, intra-decode Rayon)
SparseBlossomDecoder	region-growing sparse blossom
BeliefMatching	BP-reweighted matching
BpOsdDecoder / BPOSDDecoder	BP + ordered-statistics (qLDPC), decode_timed
LookupTableDecoder	exact table + union-find fallback
PredecodedDecoder	local predecoder + exact residual
StreamingDecoder / SlidingWindowDecoder	multi-round decoding
HybridDecoder	GNN predecoder + sparse blossom
NeuralPredecoder / GNNPredecoder / GNNTrainer	learning components
AutoDecoder	backend-routing meta-decoder
BatchDecoder / CPUBatchDecoder	CPU batch paths (SIMD / Rayon)
CUDABatchDecoder / OpenCLBatchDecoder	GPU batch paths
DecoderPool	multi-process batch decoding (auto-Rayon on Windows)
Workbench	benchmark orchestration controller

A.2 Functions, modules & utilities

Name	Purpose
generate_*_code_checks(d)	quick (check_to_qubits, n_qubits) tuples
codes	Code objects: H, logicals, distance, samplers
check_to_edges(checks)	adjacency list → edge list
decode_with_diagnostics(code, s, kind=)	decode → DecodeResult

Name	Purpose
DecodeResult	diagnostics: validity, weight, logicals, timing
get_decoder / clear_decoder_cache	LRU-cached decoder factory
get_decoder_pool	cached DecoderPool factory
decode_mmap(syndrome_path, correction_path, ...)	out-of-core memmap decoding
cuda_is_available / opencl_is_available	GPU probes
pymatching_compat / stim_compat	PyMatching & Stim bridges
sinter_compat / qiskit_plugin	Sinter & Qiskit bridges
Workbench / BenchmarkSuite	app controller & micro-benchmarks
run_mcp_server / rest_api / start_metrics_server	services (feature-gated)

Appendix B

The Standard Decoder API

Almost every decoder follows the same shape. Learn it once:

Member	Meaning
Decoder(check_to_qubits, n_qubits)	construct from the adjacency list + qubit count
decode(syndrome) -> ndarray	decode one uint8 syndrome to a correction
batch_decode(syndromes) -> ndarray	decode a 2-D array of syndromes
batch_decode_par(syndromes) -> ndarray	explicit Rayon-parallel batch (CPUBatchDecoder)
n_qubits (property)	number of qubits / columns of H
n_checks (property)	number of checks / rows of H

Exceptions and extensions are documented per decoder in Part III (for example decode_with_weights on SparseBlossom, decode_timed on BPOSDDecoder, the GPU health counters on CUDABatchDecoder, the round-based update/flush on streaming decoders, and close() on DecoderPool).

DecoderPool API (new in v0.6.3)

Member	Meaning
DecoderPool(checks, nq, decoder_type, n_workers)	construct a multi-process pool
pool.decode(syndromes) -> ndarray	decode a 2-D array (auto-Rayon on Windows)
pool.close()	release worker resources

Cached factory API (new in v0.6.3)

Function	Meaning
get_decoder(checks_tuple, nq, decoder_type)	LRU-cached decoder instance
clear_decoder_cache()	clear all cached decoders
get_decoder_pool(checks_tuple, nq, decoder_type, n_workers)	cached DecoderPool
decode_mmap(syn_path, cor_path, c2q, nq)	out-of-core memmap decode

Appendix C

Glossary

Check (stabilizer/detector)

A parity measurement; a row of H. Fires (1) when an adjacent qubit error makes its parity odd.

check_to_qubits

The adjacency list of checks to the qubits they touch — QECTOR's universal input.

Syndrome

The vector of check outcomes, s = H e (mod 2); the decoder's input.

Correction

The decoder's output: which qubits to flip back. Valid when H c = s (mod 2).

Logical operator

A row of L; an undetectable operator whose parity defines the encoded information.

Logical error

A residual r = e XOR c with L r ≠ 0 — the failure QEC tries to avoid.

Code distance (d)

The minimum weight of a logical operator; larger d tolerates more errors.

MWPM

Minimum-Weight Perfect Matching — the exact matching decoder (BlossomDecoder).

Union-Find

A near-linear-time approximate matching decoder; fast, slightly less accurate.

BP-OSD

Belief Propagation with Ordered-Statistics Decoding, for general qLDPC codes.

decode_timed

BP-OSD with a wall-clock deadline; falls back to hard-decision on timeout.

DEM

Stim's Detector Error Model — a circuit-level noise description for realistic studies.

Threshold

The physical error rate below which adding distance helps; above it, distance hurts.

DecoderPool

Multi-process batch decoder pool; auto-Rayon on Windows.

AVX2 SIMD

CPU vector instructions auto-detected for 1.1M shots/s batch throughput.

Workbench

Benchmark orchestration controller: load circuit, sweep, export report.

— End of the QECTOR Decoder v3 Official User Manual —

Covers version 0.6.3. Questions and commercial licensing: admin@qector.store